

Manuel de Configuration et Benchmarking — Groupe Web v3

Yasmine GRELE | Aicha GUEMIR | Fatima Zahra ZINAI

Benchmarks : Kernels de calcul • Suites applicatives • Mini-applications

Application Web de soumission de jobs (remplacement de l'interface obsolète)

Encadrant : MFAHCI | Mai 2026 | 4 semaines

1. Accès au cluster

Paramètre	Valeur
Compte SSH	groupe_web
Mot de passe	(communiqué par l'encadrant)
Adresse	ssh -oHostKeyAlgorithms=ssh-rsa groupe_web@41.106.128.17
Queue LSF	queue_web (UJOB_LIMIT=2, accès exclusif)
Port Flask	8081 (ou 8082, 8083 si vous lancez plusieurs instances)
Dossier de travail	/home/groupe_web/
Dossier résultats	/home/groupe_web/results/
Dossier partagé	/home/shared_project/web_interface/ (code Flask existant)
Droits sudo	Oui (groupe wheel, sans mot de passe)
Python	/home/mfahci/anaconda3/bin/python (3.8.5)
Compilateurs	gcc, g++, gfortran (via module load gcc si nécessaire)

Important

- Tous vos jobs doivent utiliser la queue queue_web (#BSUB -q queue_web).
- Vous ne pouvez pas soumettre ni tuer un job dans la queue de Bouras (queue_Bouras).
- Chaque test doit exploiter l'intégralité des 22 nœuds de calcul (352 cœurs avec 16 cœurs/nœud) sauf indication contraire.
- Ne changez jamais le mot de passe du compte partagé.

2. Construction du site web de soumission de jobs

Vous devez remplacer l'interface obsolète par une application Flask moderne capable de : - soumettre un job LSF (en passant un script ou des paramètres), - lister les jobs en cours (bjobs), - afficher les sorties (bpeek), - annuler un job (bkill).

Toutes les commandes LSF seront exécutées en **subprocess** depuis Flask. L'application tournera sur le port **8081**.

2.1 Lancer Flask

```
cd /home/shared_project/web_interface
# Lancer en avant-plan (test)
python app.py --host 0.0.0.0 --port 8081
# En arrière-plan
nohup python app.py --host 0.0.0.0 --port 8081 > flask.log 2>&1 &
# Arrêter
pkill -f 'python app.py'
```

2.2 Exemple de route Flask pour soumettre un job

```
import subprocess
from flask import Flask, request, render_template_string

app = Flask(__name__)

@app.route('/submit', methods=['POST'])
def submit_job():
    script = request.form['script']  # contenu du script LSF
    # Sauvegarder le script dans un fichier temporaire
    with open('/tmp/job_web.lsf', 'w') as f:
        f.write(script)
    # Soumettre dans la queue protégée
    result = subprocess.run(['bsub', '-q', 'queue_web', '/tmp/job_web.lsf'],
                            capture_output=True, text=True)
    return f"<pre>{result.stdout}</pre>"

# D'autres routes : /jobs, /output/<jobid>, /kill/<jobid>
# (réutilisez les commandes LSF : bjobs, bpeek, bkill)
```

Ports disponibles : vous pouvez utiliser 8081, 8082, 8083. Si un port est occupé, essayez le suivant. Aucune autre application n'est lancée sur ces ports.

3. Benchmarks Algorithmiques (Kernels de calcul)

Objectif : mesurer la puissance de calcul pure (FLOPS) sur des kernels typiques (multiplication matricielle, FFT). Tous les kernels doivent être exécutés sur **22 nœuds (352 cœurs)**. Comparer les résultats avec une exécution locale (desktop) et Google Colab (GPU T4 ou CPU seul).

3.1 Multiplication de matrices denses (DGEMM via MPI)

Script Python MPI : matrix_mpi.py

```
from mpi4py import MPI
import numpy as np, time

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

N = 8192 # taille de matrice, ajustable
# Distribution par bloc de lignes
local_rows = N // size
A_local = np.random.rand(local_rows, N)
B = np.random.rand(N, N)

start = time.perf_counter()
C_local = A_local @ B
end = time.perf_counter()

# Barrière pour mesurer le temps du plus lent
comm.Barrier()
if rank == 0:
    print(f"Multiplication {N}x{N} sur {size} rangs, temps max : {end-start:.2f} s")
```

```
gflops = (2*N**3) / (end-start) / 1e9
print(f"Performance : {gflops:.2f} GFLOPS")
```

Script LSF pour 22 nœuds : run_matrix_22.lsf

```
#!/bin/bash
#BSUB -J matrix_22
#BSUB -q queue_web
#BSUB -n 352
#BSUB -R "span[ptile=16]"
#BSUB -o /home/groupe_web/results/matrix_22_%J.out
#BSUB -e /home/groupe_web/results/matrix_22_%J.err

source /opt/ibm/platform_mpi/pmpi.sh
export LD_LIBRARY_PATH=/opt/ibm/platform_mpi/lib/linux_amd64:$LD_LIBRARY_PATH
export OMP_NUM_THREADS=1

mpirun -TCP -np 352 python /home/groupe_web/benchmarks/matrix_mpi.py
```

3.2 FFT parallèle (1D MPI)

Script : fft_mpi.py (utilise numpy.fft avec une distribution 1D).

```
from mpi4py import MPI
import numpy as np, time

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

N_global = 2**26 # ~67 millions de points
N_local = N_global // size
data = np.random.rand(N_local) + 1j*np.random.rand(N_local)

comm.Barrier()
start = time.perf_counter()
local_fft = np.fft.fft(data)
# Transposition pour obtenir un résultat global (non nécessaire si on mesure juste le calcul)
comm.Barrier()
end = time.perf_counter()

if rank == 0:
    print(f"FFT {N_global} points sur {size} rangs, temps : {end-start:.2f} s")
```

Script LSF : run_fft_22.lsf (même structure que ci-dessus, changer #BSUB -J et la commande mpirun).

Comparaisons

- Exécutez ces mêmes kernels sur Google Colab (Python pur, numpy) et sur votre PC.
 - Notez les GFLOPS et les temps pour les différentes tailles de problème.
 - Affichez les résultats dans votre dashboard Flask.
-

4. Suites Applicatives (Benchmarks de charge réelle)

Objectif : exécuter des codes simplifiés représentatifs de la météo, chimie, physique et mécanique des fluides. Chaque application doit tourner sur **22 nœuds** et produire des métriques (temps/simulation, pas de temps/seconde, etc.).

Nous fournissons des **mini-apps Python MPI** auto-contenues qui capturent les schémas de calcul et communication typiques. Vous pouvez également compiler de vraies applications HPC (voir les liens en fin de section).

4.1 Météo – Barotropic vortex (équation de Saint-Venant linéarisée)

Script : `shallow_water_mpi.py`

```
# Simulation d'un tourbillon dans un bassin carré
# Résout l'équation du tourbillon barotrope avec un schéma de semi-Lagrangien + FFT
# distribution par slabs selon Y
```

(Le script complet, ~200 lignes, est fourni par l'encadrant. Il utilise `mpi4py` et `numpy.fft`.)

Mesures : nombre de pas de temps/seconde, scaling jusqu'à 352 cœurs.

4.2 Chimie – Gray-Scott 3D réaction-diffusion

Script : `grayscott_mpi.py`

```
# Réaction-diffusion de Gray-Scott en 3D avec discrétisation différences finies
# Communication des bords entre voisins (cartésien 2D/3D)
# Paramètres : F=0.04, k=0.06, grille 2563
```

Mesures : temps par itération, cellules mises à jour par seconde.

4.3 Physique – Simulation N-corps (gravitation)

Script : `nbody_mpi.py`

```
# Calcul direct de toutes les paires ( $O(N^2)$ ) ou avec tree-code (plus réaliste)
# Version simplifiée  $O(N^2)$  pour benchmark
```

Mesures : nombre d'interactions/seconde, scaling.

4.4 Mécanique des fluides – Lattice Boltzmann 3D (D3Q19)

Script : `lbm_mpi.py`

```
# Flow in a lid-driven cavity
# Grille 1283, streaming along neighbours
# Utilise MPI pour décomposer le domaine en blocs 3D
```

Mesures : MLUPS (million de cellules mises à jour par seconde).

Chaque script est accompagné d'un fichier `run_X_22.ls` (352 cœurs, queue `queue_web`).

Références HPC (codes réels que vous pouvez installer si vous le souhaitez, nécessitent un compilateur Fortran/C) :

- MiniWeather : <https://github.com/mrnorman/miniWeather>
- Gray-Scott MPI : <https://github.com/...>

- N-body : <https://github.com/...>
- Palabos LBM : <https://github.com/...>

5. Mini- Applications Compactes (Noyaux HPC)

Ces codes combinent MPI et OpenMP pour reproduire les schémas des grandes applications.

5.1 Stencil 3D (Jacobi ou diffusion thermique)

```
# dans stencil3d_mpi_omp.py
# Chaque processus MPI exécute un bloc 3D et parallélise le noyau de stencil avec OpenMP
(Numba ou prange via Cython)
```

Si vous préférez un code C+OpenMP+MPI, l'encadrant peut vous fournir une version compilable.

5.2 Benchmark de type HPCG (Gradient Conjugué)

```
# conjgrad_mpi.py
# Résout un système creux distribué par lignes, utilise Allreduce pour les produits
scalaires
```

Tous ces benchmarks doivent être exécutés sur 22 nœuds avec 352 cœurs, en comparant les performances avec un poste local (1-16 cœurs) et Colab.

6. Calendrier 3+1 (rotation des blocs d'utilisation du cluster)

Chaque bloc de **3 jours consécutifs** est réservé à votre groupe pour des tests à pleine échelle (22 nœuds). Les jours "commun" (marqués comme transition) sont libres : pas de soumission de jobs massifs, seulement du développement ou de l'analyse.

Vue d'ensemble des blocs (Groupe Web en bleu, Bouras en orange, commun en gris)

Semaine	Lun	Mar	Mer	Jeu	Ven	Sam	Dim
1 (5-11 mai)	Web	Web	Web	Commun	Bouras	Bouras	Bouras
2 (12-18 mai)	Web	Web	Web	Commun	Bouras	Bouras	Bouras
3 (19-25 mai)	Web	Web	Web	Commun	Bouras	Bouras	Bouras
4 (26-31 mai)	Web	Web	Web	Commun	Bouras	(Fin)	–

Détail des activités sur les blocs Web

Bloc 1 – 5-7 mai (lun-mer) :

- Installation des scripts kernels et mini-apps.
- Premiers tests à petite échelle (1 nœud) pour validation, puis passage à 22 nœuds.
- Début du développement Flask (routes de soumission, dashboard).

Bloc 2 – 12-14 mai (lun-mer) :

- Exécution de tous les kernels (matrix, FFT) sur 22 nœuds, récolte des résultats.
- Suites applicatives : Gray-Scott et n-body en 22 nœuds.
- Site web : soumission fonctionnelle, affichage des jobs.

Bloc 3 – 19-21 mai (lun-mer) :

- Mini-apps stencil et HPCG en 22 nœuds.
- Application météo (shallow water) et LBM fluid.

- Intégration des résultats dans Flask (graphes comparatifs HPC vs Colab/Desktop).
- Test complet du workflow web (soumission → exécution → résultat).

Bloc 4 – 26-28 mai (lun-mer) :

- Finalisation du dashboard.
- Rédaction des analyses de scaling et comparaisons.
- Préparation de la soutenance.

Règles

- Utilisez impérativement `#BSUB -q queue_web` dans tous vos scripts.
- Ne soumettez pas de job 22 nœuds les jours “Commun” ou “Bouras”.
- Vous ne pouvez pas interagir avec les jobs de la queue `queue_Bouras`.

7. Dépannage rapide

Erreur	Cause probable	Solution
User cannot use the queue	Mauvais compte ou queue	Vérifiez <code>#BSUB -q queue_web</code> et le login <code>groupe_web</code>
UJOB_LIMIT atteint	2 jobs déjà en cours	Attendre ou <code>bkill</code> un job
MPI job fails with shm	Mémoire partagée MPI	Ajouter <code>-TCP</code> à <code>mpirun</code>
Flask port occupé	Instance précédente	<code>pkill -f 'python app.py'</code>
Fichier script sous Windows	Fins de ligne <code>\r\n</code>	<code>sed -i 's/\r\$//'</code> <code>script.lsf</code>